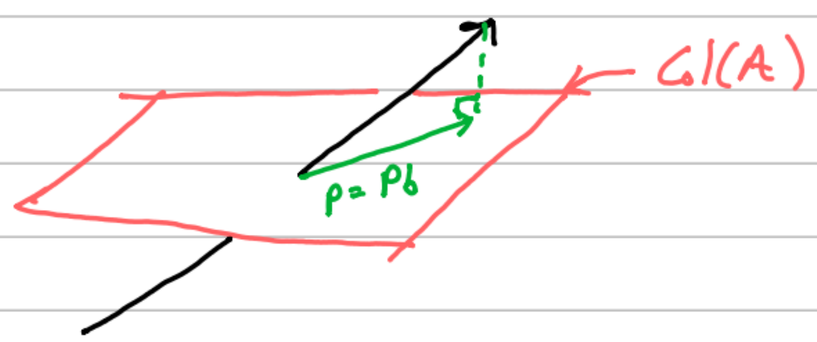
A Recap on Projections

When $Ax = b$ has no solution, we might want an approximate solution. Since there is no solution, we know that b is not in the column space of A. So we solve for the closest vector to b that is in the column space of A.

$$A\hat{x} = p$$



$p = Pb$ is the projection of b onto $Col(A)$.

Let's say $Col(A)$ has basis vectors $a_1, a_2, \dots a_k$, then $p \in Col(A)$ can be written as a linear combination of them: $p = \hat{x}_1 a_1 + \hat{x}_2 a_2 + \dots \hat{x}_k a_k$,

or

$$p = A\hat{x}$$

We also know the error vector $b - p$ must be orthogonal to $Col(A)$, that is to say, orthogonal to every vector in A.

$$a_1^T (b - A\hat{x}) = 0$$
$$a_2^T (b - A\hat{x}) = 0$$
$$\vdots$$
$$a_k^T (b - A\hat{x}) = 0$$

$$\Rightarrow A^T (b - A\hat{x}) = \underline{0}$$

$$A^T b - A^T A \hat{x} = 0$$

$$\boxed{A^T A \hat{x} = A^T b}$$

So $\hat{x} = (A^T A)^{-1} A^T b$, and so the projection vector $p = A\hat{x}$ is

$$p = A(A^T A)^{-1} A^T b$$

Meaning that the projection matrix P is

$$P = A(A^T A)^{-1} A^T$$

Linear Regression

Linear Regression is one of the foundations of machine learning. Understanding it is necessary to understand many other machine learning algorithms.

What are we trying to do?

Suppose we have a dataset of n observations. Each observation consists of d input features and a single real-valued output. For example

$$x = \begin{bmatrix} \text{height} \\ \text{weight} \\ \text{blood pressure} \\ \vdots \end{bmatrix} \in \mathbb{R}^d \qquad y = \text{chance of diabetes} \in \mathbb{R}$$

We write the ith observation as $(x^{(i)}, y^{(i)})$. Formally, our goal is to find a function $f: \mathbb{R}^d \mapsto \mathbb{R}$ such that

$$f(x) \approx y$$

for all observed pairs, and such that f generalises well to unseen data.

As the name suggests, we want to find a linear combination of parameters to achieve this (we try to fit the most suitable line / plane / hyperplane to the data)

$$f(x) = w_1 x_1 + w_2 x_2 + \dots w_d x_d + b$$

$$\Rightarrow f(x) = \underline{w}^T \underline{x} + b$$

So we have a weight vector $\underline{w} \in \mathbb{R}^d$ and a scalar bias $b \in \mathbb{R}$

The goal is to solve for the optimal w and b. To look at the problem through the linear algebra lens, in the perfect case we have

$$w_1 x_{11} + w_2 x_{12} + \dots w_d x_{1d} = y_1$$
$$w_2 x_{21} + w_2 x_{22} + \dots w_d x_{2d} = y_2$$
$$\vdots$$
$$w_n x_{n1} + w_2 x_{n2} + \dots w_d x_{nd} = y_n$$

So

$$X \underline{w} = \underline{y}$$

We find w to create the optimal linear combination of input features to form the output vector.

If it is almost certain that $y \notin Col(X)$. Then is likely no single hyperplane that passes through all data points. So, we project y onto $Col(X)$ to obtain

$$\hat{y} = X \hat{w}$$

the closest point in $C(X)$ to y. The weights $\hat{w}$ to produce this is our solution.

In practice, however, we have to solve for both the weights w and bias b. So how do we deal with this? The trick is to augment the weights vector to also include the bias, then we can solve for both in one shot.

- Append a constant 1 to every input vector

$$\tilde{x} = [x_1, x_2 \dots x_d, 1]$$

- Extend the weight vector with the bias

$$\tilde{w} = [w_1, w_2 \dots w_d, b]$$

Then:

$$\tilde{w}^T \tilde{x} = w_1 x_1 + w_2 x_2 + \dots w_d x_d + b \cdot 1$$

$$= w^T x + b$$

If we vectorize, we change

$$f(X) = X^T \underline{w} + \underline{b}$$

into

$$f(\tilde{x}) = \tilde{X} \tilde{w}$$

In general, we assume X contains the constant 1 and w contains the bias, so our prediction is:

$$\hat{y} = Xw$$

$X \in \mathbb{R}^{n \times (d+1)}$, $w \in \mathbb{R}^{(d+1)}$, $\hat{y} \in \mathbb{R}^n$

Analytical Solution

We solve via projection:

$$\hat{w} = (X^T X)^{-1} X^T y$$

We can also derive this via calculus. We use Mean Squared Error Loss

$$L(w) = \frac{1}{n} \| y - Xw \|^2$$

We want to minimise L w.r.t w

$$L(w) = \frac{1}{n} (y - Xw)^T (y - Xw)$$

scales!

$$= \frac{1}{n} (y^T y - y^T X w - w^T X^T + w^T X^T X w)$$

$$= \frac{1}{n} (y^T y - 2 w^T X^T y + w^T X^T X w)$$

Using the identities:

- $\nabla_x (a^T x) = a$
- $\nabla_x (x^T A x) = 2Ax$ when A is symmetric

and since $X^T X$ is symmetric

$$\nabla_w L(w) = \frac{1}{n}(-2X^T y + 2X^T X w)$$

Setting this to zero:

$$X^T X w = X^T y$$

$$\hat{w} = (X^T X)^{-1} X^T y$$

Computational Cost

Forming $X^T X$ is an $O(nd^2)$ operation.

Solving the linear system

$$X^T X \hat{w} = X^T y$$

Can be done in $O(d^3)$ via Gaussian elimination. This is not good when d is large (e.g $d = 10^6$ in a text embedding model). In practice, at scale we use

gradient descent, even though the Normal Equation would give an exact solution.

The Iterative Solution: Gradient Descent

Why Gradient Descent? The Normal Equation give us the exact answer in one shot, since linear regression has a convex quadratic loss with a closed-form solution. For almost every other ML model, no closed form exists - thus, we must iterate.

We start at an initial weight vector w_0 and repeatedly move in the direction of steepest descent of the loss:

$$w_{t+1} = w_t - \alpha \cdot \nabla_w L(w)$$

where $\alpha > 0$ is the chosen hyperparameter. We already derived the gradient:

$$\nabla_w L(w) = \frac{2}{n} X^T (X w_t - y)$$

Batch, Stochastic, Mini-Batch

The gradient $\nabla_w L(w)$ uses all n data points at once. Accurate, but expensive for large n.

- **Batch GD**
  Uses all n examples per step.
  Exact gradient, stable convergence
- **Stochastic GD**
  Uses one randomly chose example per step. Noisy gradient.
- **Mini-Batch GD**
  Uses a random subset of size B.
  Default in practice.

Learning Rate

Choosing α is non trivial. For linear regression, it is known that GD converges when

$$\alpha = 2\lambda_{max}$$

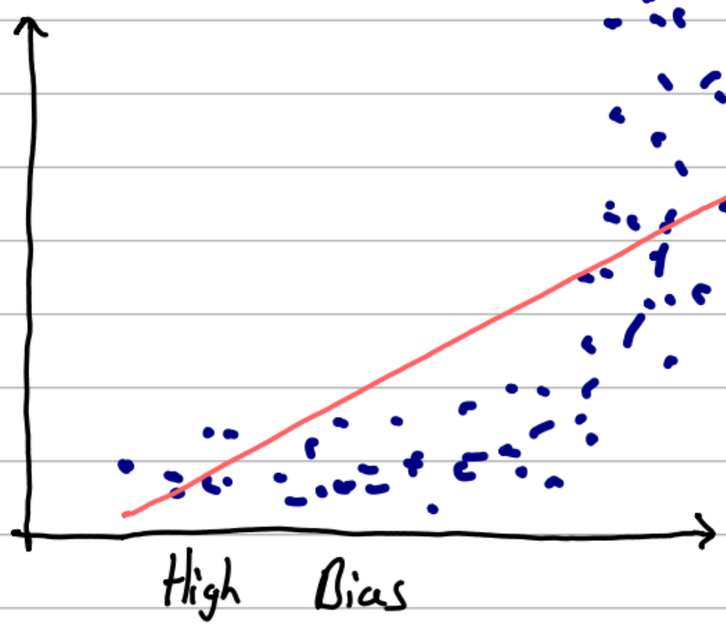
where λ_{max} is the largest eigenvalue of $\frac{2}{n} X^T X$

- α too large - overshoots minimum, loss diverges
- α too small - Convergence very slow
- α just right - Smooth convergence.

Bias/Variance Trade Off and Regularisation

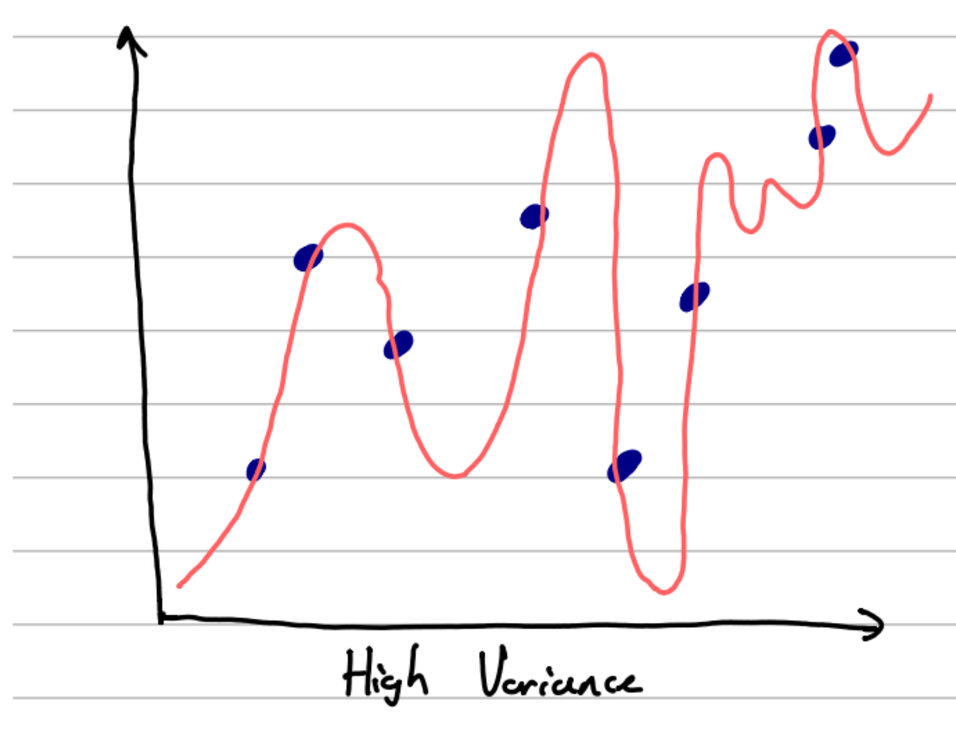
Before moving on, there are a few new concepts to cover.

Bias is a measure of the amount of error from your model on account of it being too simple. It tells you how much your chosen model matches the underlying structure of the data. Imagine trying to fit a straight line onto data defined by a quadratic curve. It doesn't matter how many data points you collect, your model is too simple to accurately fit the data - it will never be able to make good predictions, because the structures don't match.



High Bias = underfitting

Variance is a measure of the amount of error from the model being too sensitive to the data it was trained on. Imagine a set of data where the model passes through every point exactly, but zigs around wildly inbetween. If you were to apply the model to unseen data, it would be completely wrong, and if you trained it on different data from the same source, the model shape would look completely different. The model hasn't generalised



High Variance = Overfitting

Irreducible Noise is error that cannot be avoided, no matter how good the model is. This is due to randomness, noise, measurement error, unseen variable etc in the data itself.

We need to understand how Bias and Variance relate to each other. They pull in opposite directions.

As you increase your models complexity (higher order polynomials, more parameters etc), your model has more freedom to take the correct shape (lowering Bias) but increasing it too much might cause you to overfit the training data (increasing variance). Conversely, if you decrease model complexity your model will generalise better (lowering variance), but oversimplify it and the predictions will no longer be accurate (increasing bias)

Regularisation

Regularisation is any technique that deliberately constrains a model to prevent it from overfitting. It is a mechanism by which we reduce variance.

But why do we prioritise reducing variance over bias? We don't always do this, but in practice variance tends to be the bigger problem since:

- Bias is partly addressed at model selection time. When starting on ML problem, you of course choose the model you know will fit well. This means bias is probably fairly low from the get go.

- Variance is often the main enemy. If the model is genuinely too simple for the problem (high bias), regularisation makes things worse.

Formally, the expected value of the Mean Squared Error is equal to:

$$E(MSE) = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

Thus, our goal is to minimise $\text{Bias}^2 + \text{Variance}$, the best Bias/Variance trade off we can get.

One such Regularisation method is Ridge Regression

Ridge Regression

Large Weights are what allows a model to overfit. A model with large weights can produce wildly different predictions for slightly different inputs. Small changes in training data → large swings in weights → large swings in predictions. High Variance by definition. If we force the weights to stay small, the model is constrained to produce smoother, more stable predictions.

We change our loss function to penalise large weights

$$L_{ridge}(w) = \frac{1}{n}\|y - Xw\|^2 + \lambda\|w\|^2$$

Where $\lambda \geq 0$ is a hyperparameter controlling regularisation strength. If we take the gradient and set to zero:

$$\nabla_w L_{ridge}(w) = \frac{2}{n}(X^TXw - X^Ty) + 2\lambda w = 0$$

$$(X^TX + n\lambda I)w = X^Ty$$

$$\hat{w}_{ridge} = (X^TX + n\lambda I)^{-1}X^Ty$$

Compared to ordinary least squares:

$$\hat{w} = (X^TX)^{-1}X^Ty$$

So we add the λI term before inverting. This has the effect of moving the weights closer to zero than otherwise.

This increases bias, since the true weights might actually be very large, so by forcing them to be small we might be introducing systematic error, stopping the model from being able to perfectly match the data.

Model Evaluation

The R^2 value answers the question: "how much better is my model than just predicting the mean every time?"

The mean $\bar{y}$ is the simplest possible baseline, if we knew nothing about x, your best guess for any y would just be the average. R^2 compares your models performance against this baseline:

$$R^2 = 1 - \frac{\|y - \hat{y}\|^2}{\|y - \bar{y}\|^2}$$

A perfect model has $R^2 = 1$, a baseline model that always predicts the mean has $R^2 = 0$.

$R^2 < 0$ means your model is actually worse than predicting the mean.

However this metric has an issue. R^2 never decreases if you add more features, even if those features are pure noise. Adding a feature gives the model more freedom, so it can always find some way to fit the training data better, even if that improvement is meaningless. This makes R^2 bad for comparing models with different numbers of features. A model with 50 random noise features will always have a higher R^2 than a model with 5 meaningful ones, even though it is clearly worse.

Instead we use Adjusted R^2

$$R^2_{adj} = 1 - \frac{(1-R^2)(n-1)}{(n-d-1)}$$

The penalty factor $(n-1)/(n-d-1)$ grows as d increases relative to n.